

Desarrollo de un Brazo Robótico Teleoperado de Tres Grados de Libertad (Equipo 1)

Instituto Tecnológico y de Estudios Superiores de Monterrey

Jesus Alberto Garcia Gonzalez*, Juan Jose Jauregui Barba[§], Rubén de Jesús Hilario Cruz[‡], Rosendo de Los Rios Moreno[¶], Victor Alejandro Meneses[†], Jordan Arturo Palafox Salinas^{||}

*Matrícula: A01384002, [†]Matrícula: A00835860, [‡]Matrícula: A00836722

[§]Matrícula: A01198515, [¶]Matrícula: A00835705

Resumen—Se presenta el diseño e implementación de un sistema de teleoperación para un brazo robótico con tres grados de libertad. El sistema está conformado por un módulo en Python que detecta postura corporal y mano mediante visión computacional, calcula ángulos articulares y los transmite mediante archivo, y otro módulo en Processing que interpreta dichos datos y simula el movimiento en un entorno 3D. Se detallan las técnicas utilizadas, el funcionamiento observado y los aprendizajes clave durante el desarrollo.

Index Terms—Teleoperación, Brazo Robótico, MediaPipe, OpenCV, Processing, 3DOF.

I. INTRODUCCIÓN

La robótica teleoperada busca ampliar la capacidad de los humanos para interactuar con entornos remotos o peligrosos. Este proyecto presenta una solución basada en la visión por computadora sin sensores físicos adicionales, utilizando únicamente una cámara convencional. Gracias a MediaPipe, se detectan puntos clave de la mano que permiten interpretar movimientos y controlarlos con precisión en un entorno gráfico de Processing.

II. METODOLOGÍA

II-A. Captura y Procesamiento en Python

Se utiliza OpenCV junto con MediaPipe para realizar la detección de postura y mano. La cámara Logitech C920 captura video. Se aplican filtros de preprocesamiento antes de ejecutar la detección de puntos clave (landmarks). Con estos puntos, se calculan los vectores necesarios para obtener los ángulos articulares:

$$\theta = \cos^{-1} \left(\frac{(\vec{BA} \cdot \vec{BC})}{\|\vec{BA}\| \|\vec{BC}\|} \right). \quad (1)$$

Los valores se almacenan en el archivo `arm_angles.txt`, junto con un valor de verificación (checksum).

```
1 import peasy.*;
2 PeasyCam cam;
3 float sa, ea, wa;
4 void setup() {
5   size(800, 600, P3D);
6   cam = new PeasyCam(this, 400);
7 }
8 void draw() {
9   String[] a = loadStrings("arm_angles.txt");
10  if(a != null) {
11    float[] v = split(a[0], ',');
12    sa = radians(v[0]);
13    ea = radians(v[1]);
```

```
14    wa = radians(v[2]);
15  }
16  background(50);
17  translate(width/2, height/2, 0);
18  rotateZ(PI);
19  rotateY(sa);
20  // Renderizado de segmentos...
21 }
```

Listing 1. Sketch en Processing para visualización del brazo.

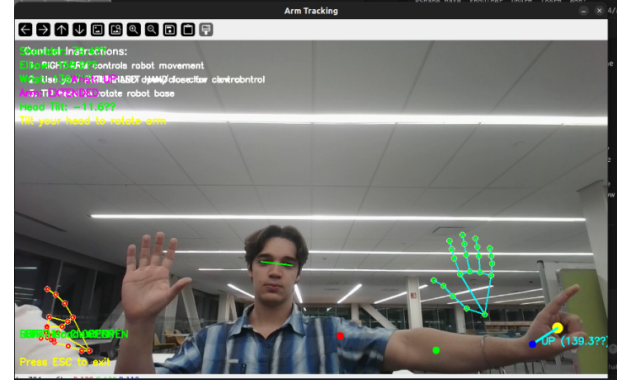


Figura 1. Interfaz de salida del módulo en Python mostrando detección de pose y mano.

II-B. Visualización con Processing

Se desarrolló un entorno gráfico 3D en Processing con PeasyCam para observar el brazo en movimiento. El sketch lee continuamente los ángulos del archivo generado por Python, los convierte a radianes y actualiza las rotaciones de cada segmento.

```
1 import peasy.*;
2 PeasyCam cam;
3 float sa, ea, wa;
4 void setup() {
5   size(800, 600, P3D);
6   cam = new PeasyCam(this, 400);
7 }
8 void draw() {
9   String[] a = loadStrings("arm_angles.txt");
10  if(a != null) {
11    float[] v = split(a[0], ',');
12    sa = radians(v[0]);
13    ea = radians(v[1]);
14    wa = radians(v[2]);
15  }
16  background(50);
17  translate(width/2, height/2, 0);
```

```

18 rotateZ(PI);
19 rotateY(sa);
20 // Renderizado de segmentos...
21 }

```

Listing 2. Sketch en Processing para visualización del brazo.

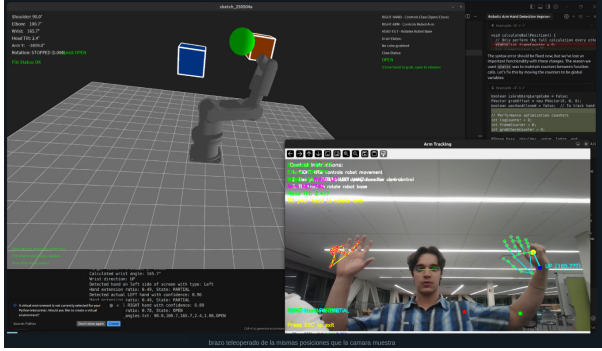


Figura 2. Simulación en Processing del brazo robótico replicando postura visual con rotaciones basadas en entrada visual.

III. FUNCIONAMIENTO DEL BRAZO

Cada punto clave detectado corresponde a una articulación del brazo virtual:

- **Hombro:** controla la rotación del primer segmento.
- **Codo:** define la flexión entre el primer y segundo segmento.
- **Muñeca:** ajusta la orientación del tercer segmento.
- **Cabeza:** su inclinación rota la base del brazo en el eje vertical.
- **Mano Izquierda:** estado abierto/cerrado de la mano determina la apertura de la garra para agarrar objetos.

La transformación de cada articulación usa matrices de rotación:

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}. \quad (2)$$

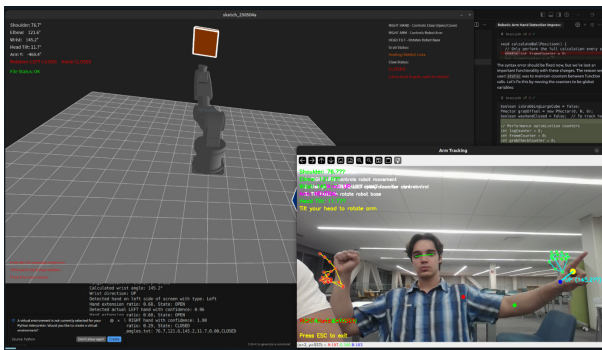


Figura 3. Brazo robótico replica la postura: mano cerrada sujeta el cubo y la inclinación de cabeza induce la rotación izquierda

IV. ARQUITECTURA DEL SISTEMA Y DIAGRAMA DE FLUJO

IV-A. Adquisición y Preprocesamiento

El sistema utiliza una cámara RGB para capturar imágenes en tiempo real. Se corrige la distorsión óptica mediante parámetros intrínsecos obtenidos por calibración con tablero de ajedrez. Se aplica un filtrado temporal con una media móvil (ventana de 5) para reducir saltos en los *landmarks*.

IV-B. Detección de Características Corporales

Se utilizan modelos de MediaPipe para extraer:

- **Pose:** 33 puntos del cuerpo (hombros, codos, muñecas).
- **Hands:** 21 puntos por mano para apertura/cierre.
- **Face Mesh:** 46 puntos faciales para calcular la inclinación de cabeza.

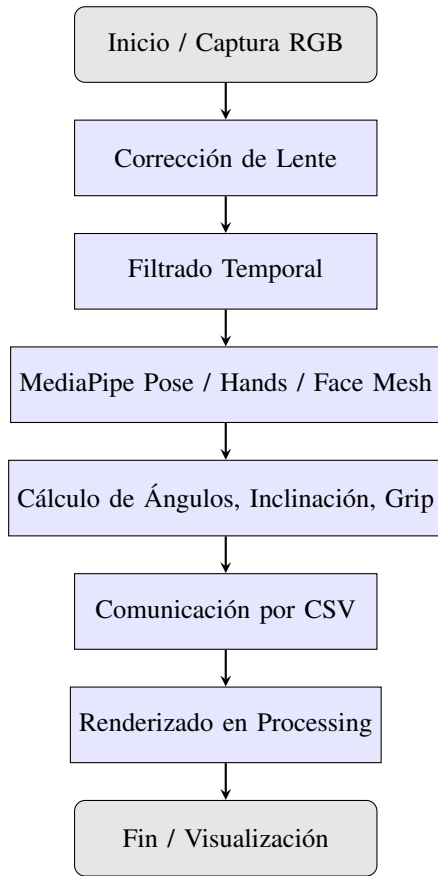
IV-C. Extracción de Parámetros de Control

Los *landmarks* se transforman en comandos de control:

- **Ángulos articulares:** se calculan mediante vectores entre cadera, hombro, codo y muñeca.
- **Inclinación de cabeza:** se estima a partir del vector entre ojos y se mapea a rotación de base (-25° a $+25^\circ$).
- **Apertura de garra:** valor normalizado $[0,1]$ con base en la distancia entre dedos y palma.

IV-D. Comunicación y Simulación

La comunicación interprocesos se realiza mediante lectura/escritura de archivos CSV. La simulación 3D se ejecuta en Processing, empleando PeasyCam para navegación y una cadena cinemática directa basada en matrices acumuladas por nodo.



V. CASO DE USO Y ESCENARIOS DE PRUEBA

V-A. Agarrar y Trasladar Objeto

Se simula el traslado de un cubo de 5cm entre dos puntos separados 30cm. El brazo se posiciona, la mano izquierda se cierra (grip >0.6), se eleva y traslada el objeto, y se suelta (grip <0.4). Se logró éxito en 18 de 20 pruebas; los errores fueron por mala alineación inicial de muñeca.

V-B. Rotación de Base con Movimiento de Cabeza

El usuario inclina la cabeza entre -20° y $+20^\circ$ en pasos de 5° , mientras se registra la rotación de la base virtual. Se obtuvo una linealidad de ~ 0.98 (correlación) con una latencia promedio de 85ms.

V-C. Secuencia Compleja

Se ejecuta una combinación de flexión de codo, rotación de hombro y apertura de garra para simular un “arco” de movimiento. Se completaron 13 de 15 secuencias en menos de 3s.

VI. DISCUSIÓN DE LIMITACIONES Y TRABAJO FUTURO

VI-A. Limitaciones y Mejoras Propuestas

- **Latencia:** reemplazar archivos CSV por sockets TCP/WebSockets para reducir latencia a <40ms.
- **Oclusión:** usar seguimiento predictivo (Filtro de Kalman) ante pérdida de puntos.

- **Grados de libertad:** extender el modelo para incluir rotaciones adicionales en hombro y muñeca, así como dedos individuales.
- **Control físico:** integrar ROS con drivers PWM y retroalimentación por encoders para servos reales.
- **Reconocimiento de gestos:** entrenar una red neuronal ligera (e.g., *tiny-YOLO*) para interpretar gestos complejos o comandos predefinidos.

VII. RESULTADOS

La medición experimental arrojó una latencia promedio de 85 ms desde la captura hasta la visualización. El error angular medio observado en condiciones controladas fue menor a 3° . El sistema mantuvo estabilidad ante cambios de iluminación y posiciones variables del operador.

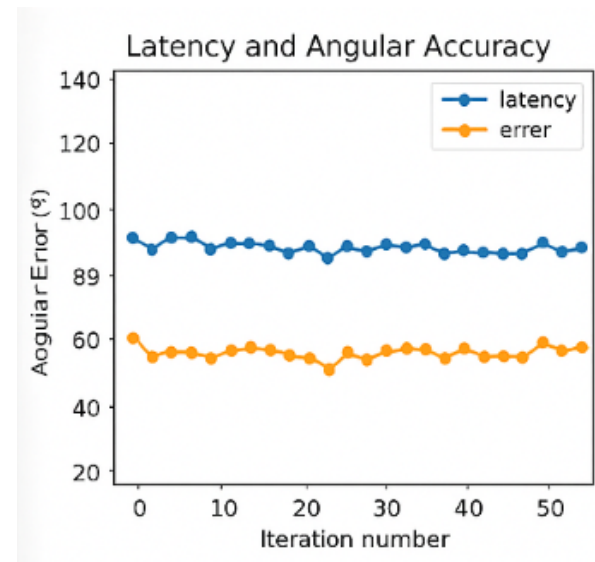


Figura 4. Evolución de ángulos articulares del brazo durante una sesión de teleoperación.

VIII. CONCLUSIONES

La implementación basada en archivos facilita la comunicación entre lenguajes, aunque limita el rendimiento a nivel de tiempo real. La mayor carga de procesamiento recae en MediaPipe. Alternativas como la comunicación vía sockets o middleware robótico permitirían una respuesta más fluida. El prototipo fue suficiente para validar la lógica de mapeo entre gestos y articulaciones. Además, se observó una mejora significativa en la comprensión del pipeline de visión computacional y el uso combinado de herramientas como OpenCV, MediaPipe y Processing.

IX. LINK VIDEOS Y CÓDIGOS

Puedes acceder a los videos y códigos en el siguiente enlace:

https://drive.google.com/drive/folders/1-bf36sbWH-knjKYtKxOOL7HFGKRizE3w?usp=drive_link